

Executable Reasoning over Query-Time Knowledge Graphs for Table–Text Question Answering

Author Names to Be Added

Affiliations and corresponding author email to be added

Abstract. Question answering over tables and text requires both identifying evidence and applying operations to that evidence. We describe Online-KG, a framework that constructs a knowledge graph for each question and uses executable candidate programs to produce answers. The design combines structural table triples with semantic triples extracted from accompanying passages. Candidate reasoning paths are translated into Python programs, executed through a restricted graph interface, and evaluated using execution outcomes and recorded evidence access. A typed numerical intermediate representation provides a complementary way to express arithmetic operations and track their inputs. We formulate an evaluation protocol on HybridQA and FinQA to examine answer quality, the contribution of graph structure and execution, and computational cost. This working draft describes the intended study; comparative benchmark results remain to be added.

Keywords: Knowledge graphs · Retrieval-augmented generation · Executable reasoning · Table question answering

1 Introduction

Tables encode relationships through row identity, column labels, and sometimes hierarchical headers. Accompanying passages can supply additional entities, events, or numeric operands. A question may require joining these sources before performing a computation. Our research question is whether a graph constructed at query time, coupled with executable reasoning and evidence-aware selection, can improve answer quality under a controlled context setting.

The proposed study investigates three components: structural table-to-graph construction, execution of multiple candidate reasoning paths, and selection using evidence access and source support. These are candidate contributions, whose empirical value and novelty require validation.

2 Related Work

To complete: review and cite primary papers on table–text QA, retrieval-augmented generation with graphs, program-aided reasoning, and execution-

based candidate selection. Compare graph construction time, evidence representation, execution interface, and evaluation setting. Do not infer novelty from an implementation difference alone.

3 Problem Formulation

Given a question q , a table T , and associated passages D , the system produces an answer a and an execution trace. In the initial benchmark setting, T and the associated passage pool are provided by the dataset. Passage ranking therefore operates within supplied context; this is not an evaluation of open-domain retrieval. The graph G_q is constructed separately for each query.

4 Method

4.1 Query-Time Graph Construction

The pipeline ranks passages before extracting evidence. Structural table triples associate a row entity with a relation derived from the column header and a cell value. Where available, hierarchical headers identify the complete column path. Semantic extraction supplies additional triples from tables and passages. The graph stores provenance with its edges and resolves entity aliases. The final paper must document the precise normalization rules and configuration.

4.2 Planning and Execution

The planner proposes candidate reasoning paths. A code synthesizer translates each path into Python that accesses a restricted graph API. Execution records queried edges and produces either an output or an error. A successful run alone does not establish that the program implements the question correctly.

4.3 Evidence-Aware Selection

The evaluator uses execution outcomes, recorded evidence, source support, and agreement between candidate outputs. Repeated access to the same source must not be interpreted as independent corroboration. Edge-access tracing indicates which evidence was consulted; it does not by itself prove that every arithmetic operand was derived from that evidence. The exact scoring rule and rejection conditions must be audited against the implementation before publication.

4.4 Typed Numerical Programs

A numerical variant represents lookup and arithmetic operations in a typed intermediate representation. The experiment will compare it with Python generation while controlling shared upstream components. Any differences in fallback behavior must be reported as part of the treatment.

5 Experimental Protocol

5.1 Datasets and Metrics

Use HybridQA for table-text QA and FinQA for numerical reasoning. Add verified dataset citations, versions, split sizes, and sample identifiers. Report HybridQA exact match and token F1. For FinQA, prioritize the official execution metric and separately label any relaxed ratio/percentage metric. Do not report Python program accuracy as equivalence to the official FinQA program metric. Keep normalized official tables and raw-table parsing experiments separate.

5.2 Baselines and Ablations

Compare direct answering from the same context, flattened-table retrieval, graph paths without execution, agreement-only selection, and the full system. Evaluate typed numerical programs separately. Planned construction ablations include structural-only and semantic-only triples and removal of header hierarchy. Verify that each ablation is implemented before running it.

5.3 Controls and Cost

Fix example identifiers, model version, decoding settings, context limits, and retrieval settings across comparable methods. Record candidate count, replanning budget, fallback policy, API attempts, logical model calls, and wall time. Use measured provider token usage if available; missing usage is not zero. Report paired uncertainty estimates and failure rates on a fixed evaluation set.

6 Results and Error Analysis

Pending benchmark evaluation. The currently inspected development artifact contains one FinQA example with execution accuracy 0, 14 logical model calls, and wall time 173.55 seconds. This is a debugging observation, not an estimate of benchmark performance. It does not contain the official execution metric. Model identity must be checked in run metadata rather than inferred from the filename. Record the repository revision for future experiments.

Analyze errors in table parsing, evidence retrieval, entity resolution, operator selection, arithmetic, units, and answer rendering. Distinguish code execution success from answer correctness. Add a case study showing the exact question, source cells, graph edges, generated program, and executed result.

7 Limitations

The initial setting assumes dataset-supplied context. Multiple model calls may increase latency. Correlated sources can weaken source-count heuristics. Restricted Python execution should not be described as a formally verified security boundary. Evidence access alone does not guarantee semantic validity.

8 Conclusion

This draft defines a study of executable reasoning over query-time table-text graphs. Final conclusions will depend on controlled benchmark and ablation results.